

halfedge：底层存储模块设计文档

src/storage.rs

halfedge 库

2026-07-04

目录

1	定位与边界	1
2	数据结构	2
2.1	三类元素	2
2.2	半边四向邻接示意	2
2.3	容器	3
2.4	SOA 位置缓存	3
3	核心 API 流程	4
3.1	新增	4
3.2	删除（自动失效）	4
3.3	访问（安全 <code>get/get_mut</code> ）	4
4	API 一览	5
4.1	容量管理方法	5
5	复杂度	6
6	单元测试覆盖	6
7	后续扩展	8

1 定位与边界

`MeshStorage` 是整个半边网格库的**底座**，只承担两类职责：

1. 三类元素（顶点 / 半边 / 面）的**存储与生命周期管理**；
2. 句柄**有效性判定**与安全访问。

它**不负责**维护任何拓扑一致性约束——`twin`、`next`、`prev`、`face` 等关系的正确性由上层构建器保证。这样切分使得底座极其稳定、可独立测试，后续可以叠加不同策略的构建器而不影响存储层。

2 数据结构

2.1 三类元素

```
1 pub struct Vertex {
2     pub position: [f64; 3],
3     pub halfedge: Option<HalfEdgeId>, // outgoing
4 }
5
6 pub struct HalfEdge {
7     pub vertex: VertexId, // (tip)
8     pub twin: Option<HalfEdgeId>,
9     pub next: Option<HalfEdgeId>,
10    pub prev: Option<HalfEdgeId>,
11    pub face: Option<FaceId>,
12 }
13
14 pub struct Face {
15     pub halfedge: Option<HalfEdgeId>, //
16 }
```

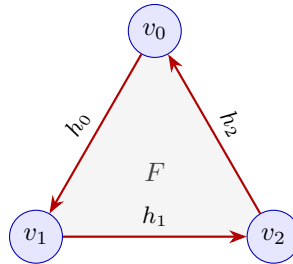
三类元素均提供 `new()` 构造函数。`Vertex` 与 `Face` 额外实现 `Default` trait:

- `Vertex::default() ≡ Vertex::new([0.0; 3])` (原点 + 无半边);
- `Face::default() ≡ Face::new()` (无半边)。

这使得二者可在需要 `Default` bound 的泛型上下文中使用 (如 `SlotMap::insert_with`、容器预分配初始化等)。`HalfEdge` 因必须指定 `vertex` 字段而无 `Default`。

2.2 半边四向邻接示意

半边 h 通过四个指针把网格「编织」起来, 下图展示一个三角面内的连接关系:



$$h_i.\text{next}: h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow h_0 \quad h_i.\text{prev}: \text{反向环} \quad h_i.\text{face} = F \quad h_i.\text{vertex} = \text{tip}$$

对应的形式化关系 (同面边界环):

$$\forall i: \quad h_i.\text{face} = F, \quad h_i.\text{next} = h_{(i+1) \bmod 3}, \quad h_i.\text{prev} = h_{(i-1) \bmod 3}, \quad h_i.\text{vertex} = v_{(i+1) \bmod 3}.$$

twin 跨面连接：若两条半边共享两 endpoint 但方向相反，则互为 twin。边界半边的 **twin** 与 **face** 通常为 **None**。

2.3 容器

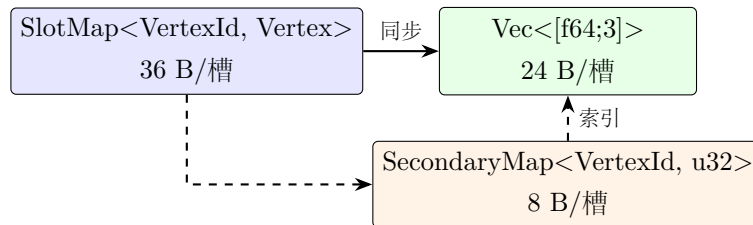
```
1 pub struct MeshStorage {
2     vertices: SlotMap<VertexId, Vertex>,
3     halfedges: SlotMap<HalfEdgeId, HalfEdge>,
4     faces: SlotMap<FaceId, Face>,
5     // SOA 2026-07
6     positions: Vec<[f64; 3]>,
7     pos_index: SecondaryMap<VertexId, u32>,
8 }
```

三类元素分别放在独立的 **SlotMap** 中，互不干扰。三组 **Id** 在编译期已被 **new_key_type!** 区分（详见 **ids.tex**），即使误把 **HalfEdgeId** 传给 **get_vertex** 也会编译失败。

2.4 SOA 位置缓存

Vertex 结构体内含 **position** (24 B) 与 **halfedge** (8 B) 两个字段，加上 **SlotMap** 槽位元数据 (4 B 版本号)，单槽步长约 36 B。几何热路径 (**mesh_volume**、**mesh_aabb**、**ray_mesh_intersects** 等) 仅访问 **position**，24 B 有效数据在 36 B 槽位中仅占 67%，造成缓存浪费。

为提升缓存命中率，**MeshStorage** 内部并行维护一份稠密的 **positions: Vec<[f64; 3]>** 缓存，步长 24 B，利用率 100%。**pos_index: SecondaryMap<VertexId, u32>** 提供 **VertexId** → **u32** 的稠密索引映射 (**SecondaryMap** 由 **Vec** 支持， $O(1)$ 查找)。



同步策略：

- **add_vertex**: 同时推入 **positions** 与 **pos_index**;
- **remove_vertex**: **swap-remove** 位置并更新填补元素的索引;
- **set_position**: 同时更新主存与缓存（推荐的位置修改入口）;
- **get_vertex_mut**: 若直接修改 **position**，缓存会暂时失效，需调用 **sync_position** 或 **rebuild_position_cache** 修复。

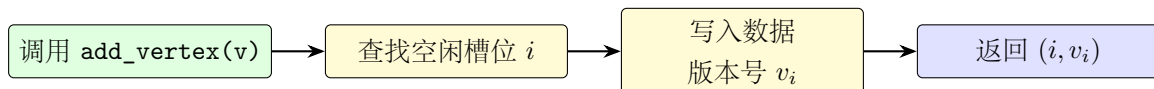
公开 API:

方法	返回	用途
<code>get_position(id)</code>	<code>Option<[f64;3]></code>	读 SOA 缓存取单点位置
<code>positions_dense()</code>	<code>&[[f64;3]]</code>	批量遍历稠密切片
<code>position_index(id)</code>	<code>Option<u32></code>	<code>VertexId</code> $\rightarrow$ 稠密索引
<code>set_position(id, pos)</code>	<code>Option<()></code>	同步更新主存与缓存
<code>sync_position(id)</code>	<code>()</code>	单点同步（修复 <code>get_vertex_mut</code> 的副作用）
<code>rebuild_position_cache()</code>	<code>()</code>	全量重建缓存（反序列化后）

收益预期：顺序遍历顶点位置时（如 `mesh_aabb`、`mesh_centroid`），缓存利用率从 67% 提升至 100%，缓存行覆盖顶点数从 $\lfloor 64/36 \rfloor = 1$ 提升至 $\lfloor 64/24 \rfloor = 2$ 。随机访问（如 `mesh_volume` 按面取顶点）的改善取决于顶点 ID 空间局部性，通常 10%–30%。

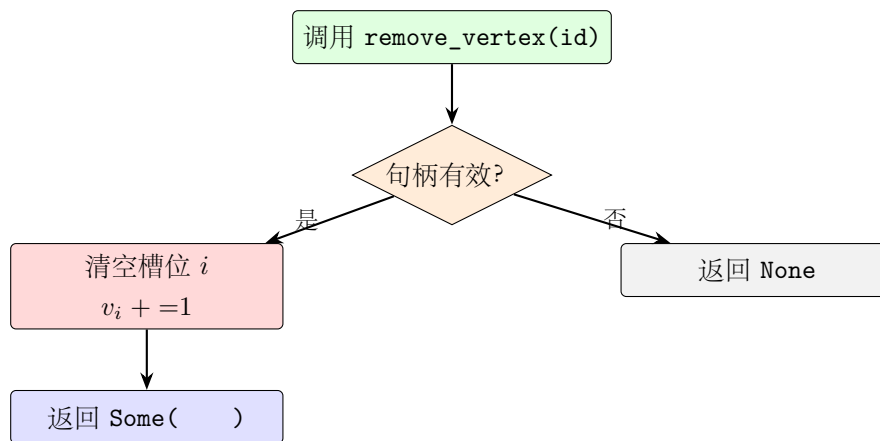
3 核心 API 流程

3.1 新增



返回值即为外部使用的强类型 ID。三条 `add_*` 接口复杂度均为 $O(1)$ 摊还。

3.2 删除（自动失效）



关键点：版本号 +1 是「自动标记失效」的核心。原句柄 $(i, v_i - 1)$ 在后续 `get` 中因版本不匹配返回 `None`；即使槽位 i 被 `add` 复用，新元素的版本是 v_i （或更高），旧句柄仍然无效，从而**杜绝 ABA 问题**。

3.3 访问（安全 `get/get_mut`）

所有访问方法返回 `Option`：

```

get_vertex(id) : VertexId  $\rightarrow$  Option<&Vertex>
get_vertex_mut(id) : VertexId  $\rightarrow$  Option<&mut Vertex>
  
```

调用方绝不会因为传入了已删除的 ID 而 panic，只会得到 None。配合 contains_* 系列，可在不确定时先做有效性预检。

4 API 一览

类别	方法	返回值
构造	new()	MeshStorage
增	add_vertex(v)	VertexId
	add_halfedge(h)	HalfEdgeId
	add_face(f)	FaceId
删	remove_vertex(id)	Option<Vertex>
	remove_halfedge(id)	Option<HalfEdge>
	remove_face(id)	Option<Face>
查（不可变）	get_vertex(id)	Option<&Vertex>
	get_halfedge(id)	Option<&HalfEdge>
	get_face(id)	Option<&Face>
查（可变）	get_vertex_mut(id)	Option<&mut Vertex>
	get_halfedge_mut(id)	Option<&mut HalfEdge>
	get_face_mut(id)	Option<&mut Face>
有效性	contains_vertex(id)	bool
	contains_halfedge(id)	bool
	contains_face(id)	bool
统计	vertex_count()	usize
	halfedge_count()	usize
	face_count()	usize
容量管理	clear()	()
	reserve(v_cap, he_cap, f_cap)	()

4.1 容量管理方法

- `clear(&mut self)`: 清空所有顶点、半边、面。等效于 `*self = MeshStorage::new().SlotMap::clear()` 是 $O(n)$ 时间但会释放内部内存，适合频繁重建网格的场景。
- `reserve(&mut self, vertex_cap, halfedge_cap, face_cap)`: 为三类 SlotMap 预分配容量，减少批量构建时的 rehash / 重分配次数。三参数均为下限提示，实际分配可能更多。`io::build_mesh_*` 构建器在入口处自动调用此方法。

5 复杂度

操作	复杂度
<code>add_*</code>	$O(1)$ 摊还（可能扩容）
<code>remove_*</code>	$O(1)$
<code>get_*</code> / <code>get_*_mut</code>	$O(1)$
<code>contains_*</code>	$O(1)$
<code>*_count()</code>	$O(1)$
<code>clear()</code>	$O(n)$ （释放内存）
<code>reserve(...)</code>	$O(n)$ （可能重分配）

6 单元测试覆盖

`src/storage.rs` 末尾的 `#[cfg(test)] mod tests` 共 54 个测试，按需求点对应如下：

测试名	覆盖点
基础 <i>CRUD</i> (3 个)	
<code>add_and_get_vertex</code>	新增 + <code>get</code> + <code>contains</code> + 计数
<code>add_and_get_halfedge</code>	半边新增 + 关联顶点 <code>outgoing</code>
<code>add_and_get_face</code>	面新增
删除与 <i>ABA</i> 安全 (4 个)	
<code>remove_vertex_invalidates_id</code>	删除后 ID 失效 + 重复删除返回 <code>None</code>
<code>remove_halfedge_invalidates_id</code>	半边删除失效
<code>remove_face_invalidates_id</code>	面删除失效
<code>slot_reuse_does_not_resurrect_old_id</code>	<i>ABA</i> 安全: 槽位复用不复活旧句柄
可变访问与无效句柄 (2 个)	
<code>get_mut_allows_in_place_update</code>	可变访问就地修改拓扑字段
<code>accessing_invalid_id_never_panics</code>	已删除 ID 访问绝不 <code>panic</code>
容量管理 (5 个)	
<code>clear_empties_all_three_slotmaps</code>	<code>clear</code> 清空三类元素 + 迭代器为空
<code>clear_equivalent_to_new</code>	<code>clear</code> 后与 <code>new()</code> 等价
<code>clear_allows_reuse_after</code>	<code>clear</code> 后仍可继续添加元素
<code>reserve_does_not_change_counts</code>	<code>reserve</code> 仅预分配, 不改变计数
<code>reserve_zero_is_noop</code>	<code>reserve(0,0,0)</code> 不 <code>panic</code>
迭代与空判断 (5 个)	
<code>is_empty_on_new_mesh</code>	新建网格 <code>is_empty()</code> 返回 <code>true</code>
<code>is_empty_after_clear</code>	<code>clear</code> 后 <code>is_empty()</code> 返回 <code>true</code>
<code>vertices_iter_yields_all_data</code>	<code>vertices()</code> 迭代所有顶点数据
<code>vertices_mut_allows_modification</code>	<code>vertices_mut()</code> 允许就地修改
<code>empty_iterators_yield_nothing</code>	空网格所有数据迭代器不产出
<i>SOA</i> 位置缓存 (9 个, 2026-07 新增)	
<code>soa_cache_get_position_matches_vertex</code>	<code>get_position</code> 与主存一致
<code>soa_cache_positions_dense_length_matches</code>	稠密切片长度与顶点数一致
<code>soa_cache_position_index_round_trip</code>	<code>VertexId</code> → 索引 → 位置往返
<code>soa_cache_set_position_syncs_both</code>	<code>set_position</code> 同步主存与缓存
<code>soa_cache_remove_preserves_dense_layout</code>	删除后稠密布局保持正确
<code>soa_cache_sync_position_after_get_vertex_mut</code>	<code>sync_position</code> 修复缓存失效
<code>soa_cache_rebuild_from_master</code>	<code>rebuild_position_cache</code> 全量重建
<code>soa_cache_clear_empties_cache</code>	<code>clear</code> 同步清空缓存
<code>soa_cache_empty_mesh_no_panic</code>	空网格访问缓存不 <code>panic</code>
<code>soa_cache_icosphere_consistency</code>	<code>icosphere</code> 全顶点缓存一致性

全库 `cargo test` 当前共 669 个单元测试 + 24 个集成测试，全部通过。

7 后续扩展

存储层刻意保持极简，后续模块基于此构建了完整能力：

- **拓扑操作** (`topology_ops.rs`): 实现 `split_edge`、`flip_edge`、`collapse_edge`、`extrude_face`、`add_triangle` 等高级操作，负责维护 `twin/next/prev` 一致性；
- **遍历器** (`traversal.rs`): 基于 `next/prev` 环遍历面边界，基于 `twin` 跨面遍历邻接面，提供 `eager/lazy` 两类迭代器与无向边迭代器；
- **序列化** (`io.rs`): OBJ/PLY 加载与保存，支持 `n-gon`；
- **并行**: `SlotMap` 内部槽位互相独立，方便后续做并行构建。